FTE 4312 | FINAL PRESENTATION

Deep Learning-Based Delta-Neutral Statistical Arbitrage on Perpetual Funding Rates

A blockchain-first presentation of our hybrid prototype: off-chain research generates the strategy view, while an on-chain vault handles deposits, withdrawals, share accounting, strategy state, and NAV or PnL synchronization.

MEMBER 1
Jie Wen

MEMBER 2
Qihang Han

MEMBER 3
Hongrui Pan

BLOCKCHAIN PROJECT FRAMING

Off-chain intelligence, on-chain state machine

The current artifact set contains a positive post-cost out-of-sample strategy under the configured assumptions.

Off-chain package areas **14**	Python scripts **16**
Prepared contract calls **2**	Contract tests **15**

Presentation-ready blockchain showcase with contract workflow, research evidence, and live demo links.

EXECUTIVE OUTLINE

System Architecture & Project Deliverables

SECTION 1

Project idea

Frame the project as a blockchain vault problem: how to represent strategy participation, accounting, and updates transparently on-chain.

SECTION 2

Technical design

Show the hybrid architecture: off-chain analytics produce strategy outputs, while a Solidity vault stores the actionable state and accounting.

SECTION 3

Implementation

Walk through the vault contract, role model, update scripts, Foundry tests, and the frontend bridge to the demo snapshot.

SECTION 4

System demo

Use this presentation page first, then move into the interactive dashboard only when you need deeper drill-down.

BLOCKCHAIN SCOPE

Bridging Off-Chain Analytics with On-Chain Vaults

Vault asset	mUSDC
Asset decimals	6
Chain label	demo_showcase_local
Role model	owner + operator
Generated	Apr 20, 2026, 04:56 PM

Core Value Proposition

- The blockchain contribution is the vault accounting boundary, not on-chain execution of the whole strategy.
- Off-chain model outputs are packaged into explicit strategy-state, NAV, and PnL updates rather than hidden inside the frontend.
- Owner and operator roles are separated so the demo can talk about governance and operational trust assumptions.
- Foundry scripts and tests make the chain-facing workflow reproducible for local demonstration.

PROBLEM AND MOTIVATION

Why this is a blockchain systems problem, not only a trading idea

While finding statistical alpha is difficult, pooling capital to execute it introduces a major trust boundary. The core infrastructure challenge is ensuring that off-chain trading states and NAV updates remain cryptographically transparent to on-chain depositors without running heavy analytics directly on the EVM.

CHALLENGE

Pooled strategy capital needs transparent accounting

Even if the strategy logic is off-chain, user deposits, share balances, NAV, and withdrawals benefit from deterministic on-chain bookkeeping.

CHALLENGE

Smart contracts cannot natively see market data or ML outputs

This creates the oracle-style boundary in our system: signals and reports are computed off-chain, then synchronized to the vault through an explicit updater path.

CHALLENGE

Role separation matters for safety and operability

The vault must distinguish owner privileges, operator privileges, and user actions, rather than treating every state change like a single admin shortcut.

CHALLENGE

A blockchain course project should demonstrate more than charts

The most defensible demo is an end-to-end flow where research outputs become explicit contract updates, events, and user-facing state.

SYSTEM BOUNDARY

What stays off-chain

OFF-CHAIN

Market data and feature pipeline

Historical market data fetching, canonicalization, feature engineering, and label construction are all implemented in Python modules and scripts.

OFF-CHAIN

Model training and ranking

Baseline models, deep-learning comparisons, signal generation, and backtests stay off-chain because they are data-heavy and iterative.

OFF-CHAIN

Research artifacts and reporting

Trade logs, equity curves, robustness tables, charts, and presentation assets are generated before any contract update is attempted.

OFF-CHAIN

Update packaging for the vault

Off-chain outputs are compressed into a small update payload: strategy state, NAV/PnL numbers, and signal/metadata/report hashes.

SYSTEM BOUNDARY

What moves on-chain

ON-CHAIN

User funds and shares

The vault receives deposits and withdrawals and maps them to internal non-transferable shares rather than a simple balance log.

ON-CHAIN

Role-restricted updates

Only the owner or operator can update strategy state, NAV, and PnL, which makes the trust boundary explicit.

ON-CHAIN

Persistent strategy state

The contract stores strategy state, reported NAV, cumulative PnL, timestamps, and last signal/metadata/report hashes.

ON-CHAIN

Safety and governance controls

Pause control, operator management, and two-step ownership transfer make the on-chain layer look like a real vault prototype rather than a static mock.

OFF-CHAIN DEEP DIVE

What happens before the contract is touched

This page is specifically about the part the chain does not do: data engineering, modeling, evaluation, and packaging a vault update that is small enough to push on-chain.

OFF-CHAIN WORKFLOW

Lifecycle before a contract update

1. Build research context

Python modules fetch market data, align timestamps, engineer features and labels, and define the candidate strategy universe.

2. Train and compare models

Model ranking, signal generation, and strategy evaluation happen off-chain because they require iteration over large datasets and repeated experiments.

3. Produce evidence

Leaderboards, trade logs, equity curves, robustness tables, and charts are generated before any state is synchronized to the vault.

4. Reduce to a compact payload

The off-chain side compresses a rich research result into a small update package that a contract can actually store: state, NAV/PnL, and hashes.

OFF-CHAIN IMPLEMENTATION

Implemented off-chain surfaces

Python package surface

The repository already has dedicated packages for data, features, labels, models, signals, backtest, evaluation, integration, and reporting.

14 package areas

Workflow scripts

Runnable scripts already cover fetching market data, building features and labels, training models, generating signals, running backtests, and exporting demos.

16 scripts

Outputs handed to the chain side

The chain side does not receive the full dataset; it receives distilled state and hashes derived from these off-chain artifacts.

state + NAV/PnL + hashes

ON-CHAIN DEEP DIVE

What the vault stores and enforces on-chain

This page focuses on the Solidity contract itself: user actions, updater permissions, persisted state, and the safety controls that make it look like a real vault prototype.

ON-CHAIN MECHANICS

Functions exposed by the vault

User-facing functions

`deposit` and `withdraw` turn asset movements into internal shares and back into assets under reported NAV constraints.

Updater-facing functions

`updateStrategyState`, `updateNav`, and `updatePnl` move off-chain strategy and valuation information into contract state.

Admin safety functions

`setOperator`, `pause`, `unpause`, `transferOwnership`, and `acceptOwnership` handle governance and emergencies.

Events and audit trail

Deposits, withdrawals, strategy-state updates, NAV updates, PnL updates, and operator changes are all emitted as events for traceability.

ON-CHAIN STATE

What is actually persisted in storage

Economic state

The vault persists `totalShares`, per-user `shareBalance`, and `reportedNavAssets` as the accounting backbone.

shares + NAV

Strategy state

The contract stores `StrategyState` with enum values `Idle`, `Active`, `Emergency`, and `Settled`.

4 enum states

Update traceability

The latest signal, metadata, and report hashes plus update timestamps are stored to anchor where the on-chain state came from.

hashes + timestamps

Access and pause state

The vault keeps owner, pending owner, operator, and paused state so operational control is visible and enforceable on-chain.

owner/operator/pause

BOUNDARY HANDOFF

What crosses from off-chain analytics into on-chain state

The most important boundary is not “everything stays off-chain” or “everything goes on-chain”; it is the exact, explicit payload that crosses between them.

PAYLOAD FIELDS

Exact things that cross the boundary

Strategy state

A compact enum indicating whether the strategy is idle, active, emergency, or settled.

`updateStrategyState(StrategyState, signalHash, metadataHash)`

Reported NAV

A scalar valuation number that mirrors the off-chain view of vault assets without pushing the full valuation model on-chain.

`updateNav(newReportedNavAssets, reportHash)`

PnL delta

A signed delta that adjusts reported NAV and cumulative PnL without replaying the full backtest path inside the contract.

`updatePnl(pnlDeltaAssets, reportHash)`

Signal / metadata / report hashes

Hashes act as lightweight anchors to richer off-chain artifacts and make the boundary explicit instead of hiding provenance in the UI.

`bytes32 signalHash / metadataHash / reportHash`

BOUNDARY SEQUENCE

How the handoff works in practice

Research output selected

The off-chain engine picks a strategy state and valuation update worth synchronizing.

Updater script configured

Environment variables specify whether to update strategy state, NAV, PnL, or a combination of them.

Contract call broadcast

Foundry broadcasts only the small set of parameters the vault actually needs to persist.

New on-chain state visible

Users and the frontend can read the updated vault state without re-running the off-chain research pipeline.

WHY THIS SPLIT

Why we do not push everything on-chain

- This boundary keeps gas costs and contract complexity small.
- It makes trust assumptions visible: the updater path is explicit and role-restricted.
- It avoids pretending that model training or backtesting belongs on-chain.
- It still gives the vault a real blockchain role: funds, shares, state, and event history live on-chain.

IMPLEMENTATION STATUS

How far the implementation has actually gone

This slide is meant to answer the practical question directly: which parts are already implemented, which parts are prototype-level, and which parts are intentionally left out.

IMPLEMENTATION EXTENT

Current project boundary

Off-chain analytics engine

IMPLEMENTED

Python package areas already cover data, features, labels, models, signals, backtest, evaluation, reporting, demo, and integration.

14 package areas

Vault contract layer

IMPLEMENTED

The Solidity side already includes the mock asset, vault contract, update paths, events, pause logic, and ownership mechanics.

2 contracts

Automation and validation

IMPLEMENTED

Local scripts deploy and update the vault, while Foundry tests validate the most important accounting and access-control behavior.

2 scripts + 15 tests

Frontend bridge

PROTOTYPE

The dashboard and presentation page already surface prepared contract-call context, but the operator experience is still demo-oriented.

demo-ready

Production-grade infra

FUTURE

Oracle verification, automated relayers, multi-asset support, audited deployment, and live exchange execution are intentionally not finished.

out of scope

OFF-CHAIN IMPLEMENTATION

What is already built outside the chain

Data + preprocessing

Implemented under `src/funding_arb/data`, `features`, and `labels` with runnable scripts for fetching data and building pipeline inputs.

16 Python workflow scripts

Modeling + signal generation

Implemented under `models` and `signals`; includes baseline training, DL comparison, and standardized signal export.

baseline + DL + signals

Backtest + evaluation

Implemented under `backtest` and `evaluation`; produces leaderboards, trade logs, equity curves, and robustness outputs.

cost-aware metrics

Reporting + demo export

Implemented under `reporting`, `demo`, and `demo_showcase`; exports the frontend-ready JSON plus chart assets you are presenting now.

dashboard + pre page

ON-CHAIN IMPLEMENTATION

What is already built on the Solidity side

MockStablecoin.sol

Implemented mint, approve, transfer, and transferFrom so the local demo can walk through approval and vault deposit without external token dependencies.

asset simulation ready

DeltaNeutralVault.sol

Implemented deposit, withdraw, share math, state updates, NAV/PnL sync, pause control, operator management, and two-step ownership transfer.

4 strategy states

Foundry scripts

Implemented `DeployLocal.s.sol` and `UpdateVaultState.s.sol` to make the chain demo runnable from deployment to updater flow.

2 scripts

Foundry tests

Implemented tests for constructor state, deposit, withdraw, pause, operator permissions, NAV/PnL updates, and ownership transfer.

15 tests

BLOCKCHAIN EVIDENCE

What proves this is more than a report with charts

The quant result stays in the loop, but the presentation emphasis moves to contract capabilities, updater flow, and local validation of the vault mechanics.

BLOCKCHAIN-CENTERED TAKEAWAY

The vault is the core system artifact

The research pipeline still matters, but the blockchain value of this project is the contract layer that turns off-chain output into explicit deposits, shares, state transitions, and NAV or PnL updates.

STRATEGY STATE

active

REPORTED NAV

110,199,990,715

PREPARED CALLS

2

STRICT RETURN

10.20%

- The chain-facing demo is about accountability and update flow, not pretending the whole strategy is fully on-chain.

- Owner/operator updates, pause controls, and explicit share accounting are the most important blockchain mechanisms to highlight.
- The strategy metrics remain supporting evidence that the off-chain engine is meaningful enough to justify a vault layer.

Research support in one line

Best strict model	TransformerEncoder
Strict Sharpe	1.584
Strict net PnL	$10,199.99

CONTRACT CAPABILITIES

What the Solidity layer actually implements

CAPABILITY	PURPOSE	EVIDENCE
Deposit / Withdraw	Let users enter and exit the vault against internal shares.	Validated via `Deposit/Withdraw` events. Uses `_ceilDiv` math to protect share price against rounding exploits.
Owner / Operator split	Separate governance authority from routine update authority.	Enforced by `onlyOwnerOrOperator` modifier, utilizing `Ownable2StepLite` for secure governance handoff.
Strategy-state updates	Record whether the system is Idle, Active, Emergency, or Settled.	Anchors off-chain intent by emitting `StrategyStateUpdated` with explicit `bytes32` signal and metadata hashes.

CAPABILITY	PURPOSE	EVIDENCE
NAV / PnL synchronization	Mirror off-chain valuation and strategy outcome into contract state.	Mirrors valuation on-chain by emitting `NavUpdated`, permanently anchoring the backtest `reportHash`.
Pause + ownership transfer	Provide emergency controls and cleaner admin handoff.	Uses `whenNotPaused` modifier to halt user flows during emergencies, protecting the vault from extreme basis dislocations.

VALIDATION AND ASSURANCE

Why the chain-facing demo is credible

CONTRACT TESTS

15

Foundry tests cover share accounting, pause controls, update authorization, and ownership flow.

PREPARED CALLS

2

The frontend demo bundle already includes a prepared contract-call summary for vault updates.

HASH-ANCHORED FIELDS

3

Signal, metadata, and report hashes help explain how off-chain evidence is referenced on-chain.

ROLE MODEL

owner + operator

The demo can discuss governance trust assumptions instead of pretending everything is trustless.

Deep Learning TransformerEncoder	**10.20%** supporting research context
Rule-Based Baseline Rule-Based Baseline	**2.40%** supporting research context
Simple ML Baseline Baseline ML	**4.10%** supporting research context

LIVE VAULT CONSOLE

Operate the vault during the presentation

This is a local presentation simulator of the Solidity vault workflow. Use it live to show how deployment, deposits, strategy updates, NAV synchronization, and withdrawals change visible contract-facing state.

PRESENTATION OPERATOR

Local chain-style vault operations

LOCAL SIMULATOR

Click these in order during the presentation. Each action mutates the visible vault state, creates a transaction hash, and appends an event just like the contract-facing flow you would inspect after a local Foundry run.

Deploy local vault

DeployLocal.s.sol

Deposit 10,000 mUSDC

approve + deposit

Sync strategy state

updateStrategyState

Sync NAV / PnL

updateNav

Withdraw 2,500 mUSDC
withdraw

Reset
start over

Deploy
contracts ready

Deposit
shares minted

Strategy
state synced

NAV
PnL reflected

Withdraw
shares burned

RESULT

Ready. Start with local deployment.

LAST TX

pending

VAULT

pending

ON-CHAIN READABLE STATE

Vault state after each click

Strategy state	idle
Selected strategy	not selected
Direction	flat
Reported NAV	$100,000.00
Vault cash	100,000 mUSDC
User wallet	25,000 mUSDC
User shares	0
Total shares	100,000
Share price	$1.00
Cumulative PnL	$0.00

EVENT STREAM

What the audience can verify

ConsoleReady

Local vault simulator loaded from the presentation demo snapshot.

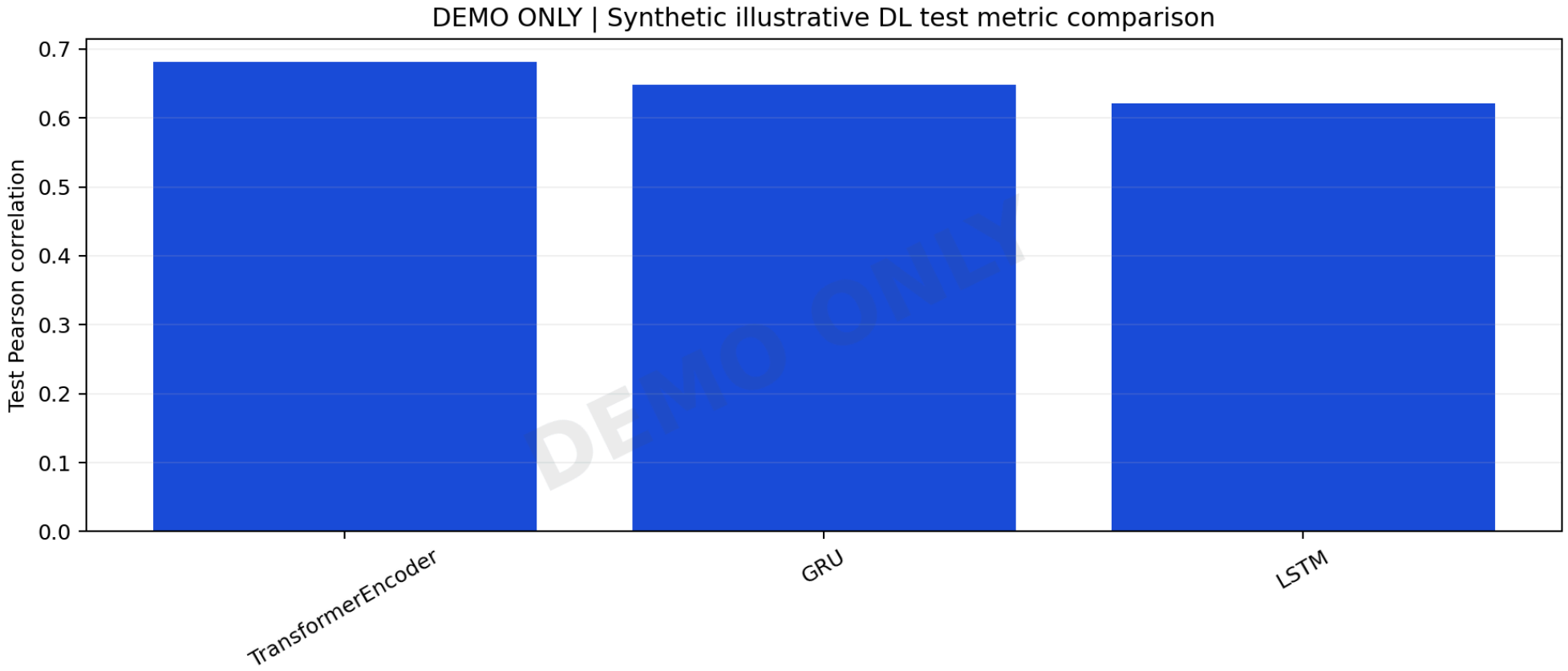
block pending

`pending`

OFF-CHAIN INTELLIGENCE

What statistical edge justifies the on-chain vault?

Before pooling capital in a smart contract, the underlying strategy must prove resilient against execution frictions. These analytics serve as the statistical foundation for our oracle updates.

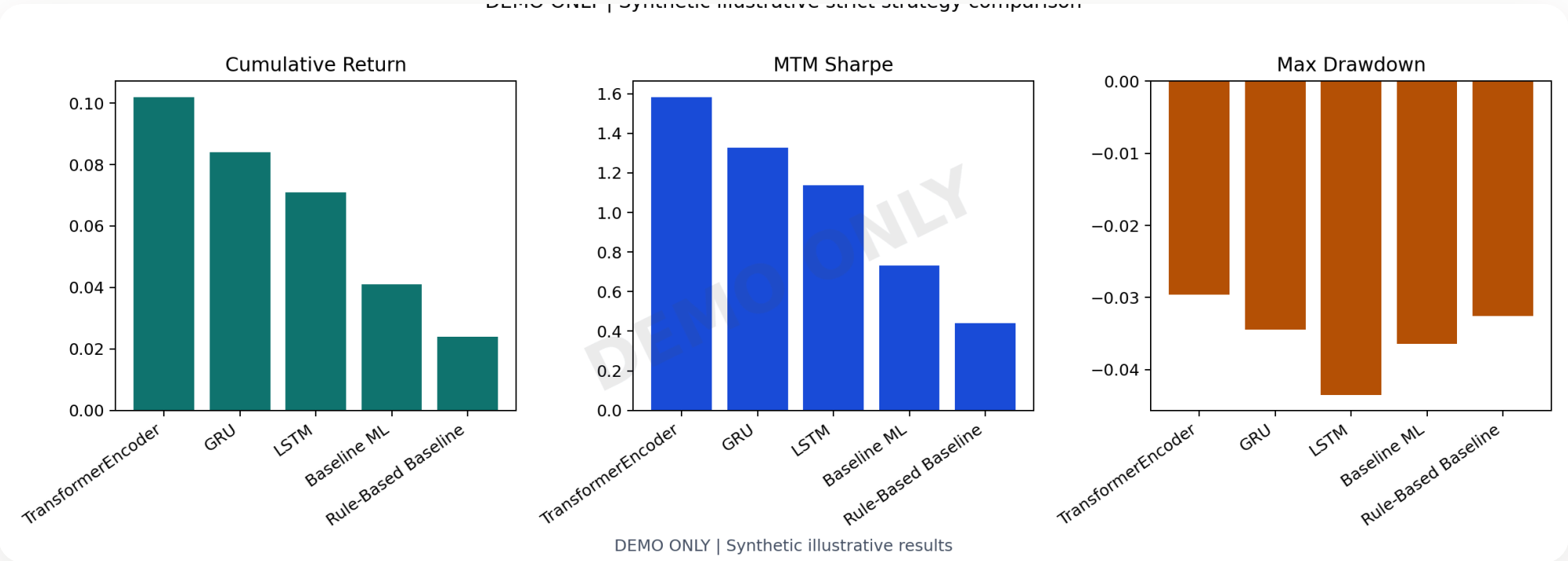


DEMO ONLY | Synthetic illustrative results

MODELS

DL test metric comparison

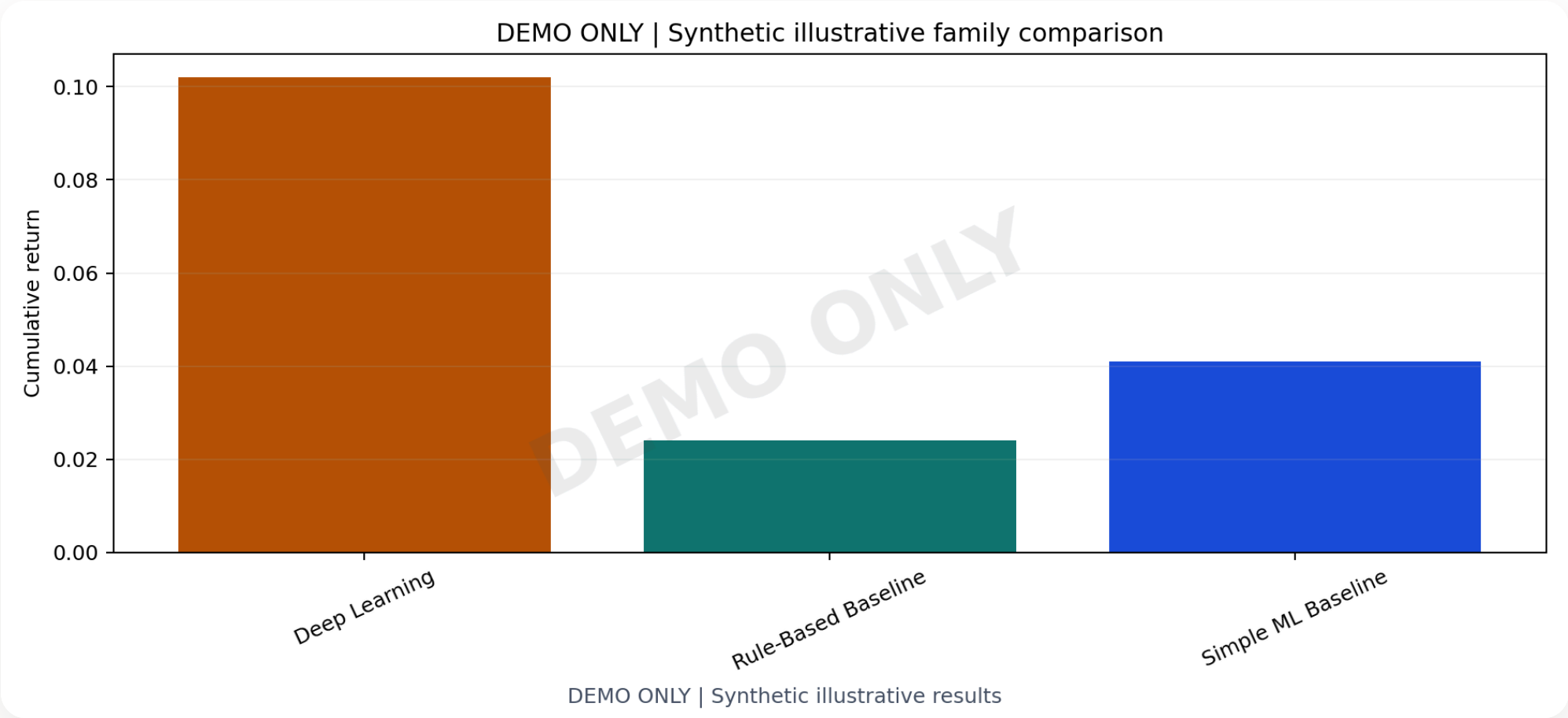
Synthetic illustrative results: LSTM, GRU, and Transformer improve on the baseline with visible but not absurd separation.



MODELS

Strict strategy comparison

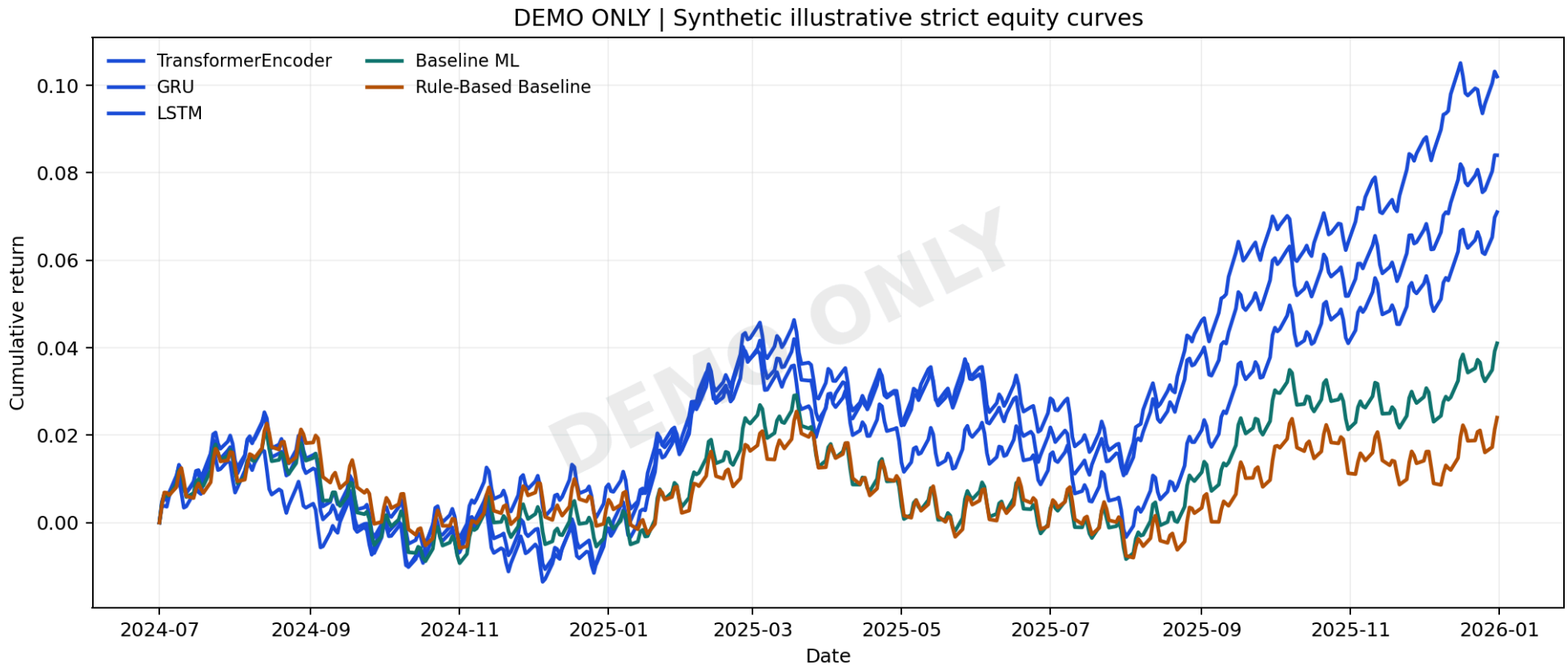
Synthetic illustrative results: one model leads on return, one on Sharpe, and one on drawdown control.



ROBUSTNESS

Family comparison

Synthetic illustrative results: rule-based, baseline ML, and DL families keep a believable ordering without turning into a perfect holy grail.



DEMO ONLY | Synthetic illustrative results

BACKTEST

Strict equity curves

Synthetic illustrative results: the baseline grinds higher, the DL curves lead, and each one still experiences visible setbacks.

CHAIN DEMO FLOW

Steps to demonstrate the vault locally

1. DEPLOY LOCAL DEMO CONTRACTS

1. Deploy local demo contracts

Use `DeployLocal.s.sol` to deploy `MockStablecoin` and `DeltaNeutralVault` into a local/test network environment.

2. FUND AND APPROVE

2. Fund and approve

Mint `mUSDC` to demo users, approve the vault, and show how assets become shares through the deposit function.

3. ACTIVATE STRATEGY STATE

3. Activate strategy state

Call `updateStrategyState` as owner/operator and explain the strategy-state enum plus signal and metadata hashes.

4. SYNC NAV OR PNL

4. Sync NAV or PnL

Run `UpdateVaultState.s.sol` with explicit toggles for `UPDATE_NAV` and `UPDATE_PNL`, together with a report hash.

5. WITHDRAW AND INSPECT ACCOUNTING

5. Withdraw and inspect accounting

Show how withdrawals depend on shares, reported NAV, and actual token liquidity instead of a naive balance subtraction.

VAULT DEMO STORY

How the contract records strategy intent and accounting

Chain label	demo_showcase_local
Vault Contract	`DeltaNeutralVault.sol`
Vault Asset	mUSDC (Decimals: 6)
Share model	internal non-transferable shares
Strategy State	ACTIVE
Selected strategy	transformer_encoder
Reported NAV	$110,199.99

Summary PnL	$10,199.99
Prepared calls	2

The key demo message is that state changes are explicit and parameterized: the update script toggles strategy-state, NAV, and PnL updates separately and can carry signal, metadata, and report hashes into the contract-facing flow.

CAVEATS

What the boundaries and limitations of this prototype are

Executive Summary

- This presentation treats the vault and updater flow as the center of the project, with the research pipeline as the off-chain engine behind it.
- The strategy is not fully executed on-chain; instead, the contract provides transparent accounting, role-based updates, and user-facing state.
- Supporting research figures remain available, but the strongest blockchain contribution is the Solidity vault prototype and its local validation workflow.

Limitations

- The vault relies on a trusted owner/operator path; it does not yet verify oracle data or signatures on-chain.
- Execution remains off-chain, so the contract mirrors strategy state and accounting rather than performing live multi-venue trading.
- The demo uses a single mock stablecoin and a simplified local/testnet flow rather than production-grade token, oracle, and fee infrastructure.
- This is a course-project prototype and not an audited DeFi system.

NEXT STEPS

What we built and where it can go next

Key Contributions

- A readable Solidity vault that models deposit, withdrawal, internal share accounting, and strategy state for a quant strategy prototype.
- A role-based bridge from off-chain signals to on-chain state using owner/operator updates plus signal, metadata, and report hashes.
- A lightweight local blockchain workflow built with Foundry scripts and tests rather than a purely static DeFi mockup.
- A frontend demo that connects research outputs and contract-facing state so the system can be presented end to end.

Future Work

- Add signed or oracle-mediated update verification so NAV and signal updates are less trust-heavy.
- Introduce fee accounting, withdrawal queue logic, and richer role controls for a more realistic vault lifecycle.
- Extend from a single mock asset to multi-asset collateral and more realistic on-chain accounting around execution costs.
- Connect local update scripts to a cleaner operator dashboard for repeatable demo and governance workflows.